

ЛАБОРАТОРНАЯ РАБОТА №10 (2+2+1+2+3 БАЛЛОВ)

ОДНОСВЯЗНЫЕ СПИСКИ

Вариант	Задача 1	Задача 2	Задача 3	Задача 4	Задача 5
1	Dynamic3	Dynamic24	ListWork1	ListWork22	ListWork59
2	Dynamic4	Dynamic23	ListWork2	ListWork23	ListWork60
3	Dynamic5	Dynamic22	ListWork3	ListWork24	ListWork61
4	Dynamic6	Dynamic21	ListWork4	ListWork25	ListWork62
5	Dynamic7	Dynamic20	ListWork5	ListWork22	ListWork59
6	Dynamic8	Dynamic19	ListWork6	ListWork23	ListWork60
7	Dynamic9	Dynamic18	ListWork7	ListWork24	ListWork61
8	Dynamic10	Dynamic17	ListWork8	ListWork25	ListWork62
9	Dynamic11	Dynamic16	ListWork9	ListWork22	ListWork59
10	Dynamic12	Dynamic15	ListWork10	ListWork23	ListWork60
11	Dynamic13	Dynamic14	ListWork11	ListWork24	ListWork61
12	Dynamic3	Dynamic25	ListWork12	ListWork25	ListWork62
13	Dynamic4	Dynamic26	ListWork13	ListWork22	ListWork60
14	Dynamic5	Dynamic27	ListWork14	ListWork23	ListWork61
15	Dynamic6	Dynamic28	ListWork15	ListWork24	ListWork62

Для каждого варианта необходимо спроектировать и реализовать класс, моделирующий заданную динамическую структуру данных (например, стек, очередь, список, ...). Класс должен содержать необходимые конструкторы (по умолчанию, копирования) и деструктор. Данные класса должны быть инкапсулированы. Программа должна демонстрировать создание объектов, выполнение операций (добавление, удаление, поиск) и решение задачи варианта.

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ: СТЕК

В заданиях данной подгруппы структура «стек» (stack) моделируется цепочкой связанных узлов-записей типа TNode. Поле Next последнего элемента цепочки равно nullptr. Вершиной стека (top) считается первый элемент цепочки. Для доступа к стеку используется указатель на его вершину (для пустого стека данный указатель полагается равным nullptr). Значением элемента стека считается значение его поля Data.

Dynamic3. Дано число D и указатель P1 на вершину непустого стека. Добавить элемент со значением D в стек и вывести адрес P2 новой вершины стека.

Dynamic4. Дано число N (> 0) и набор из N чисел. Создать стек, содержащий исходные числа (последнее число будет вершиной стека), и вывести указатель на его вершину.

Dynamic5. Дан указатель P1 на вершину непустого стека. Извлечь из стека первый (верхний) элемент и вывести его значение D, а также адрес P2 новой вершины стека. Если после извлечения элемента стек окажется пустым, то положить $P2 = \text{nullptr}$. После извлечения элемента из стека освободить память, занимаемую этим элементом.

Dynamic6. Дан указатель P1 на вершину стека, содержащего не менее десяти элементов. Извлечь из стека первые девять элементов и вывести их значения. Вывести также адрес новой вершины стека. После извлечения элементов из стека освобождать память, которую они занимали.

Dynamic7. Дан указатель P1 на вершину стека (если стек пуст, то $P1 = \text{nullptr}$). Извлечь из стека все элементы и вывести их значения. Вывести также количество извлеченных элементов N (для пустого стека вывести 0). После извлечения элементов из стека освобождать память, которую они занимали.

Dynamic8. Даны указатели P1 и P2 на вершины двух непустых стеков. Переместить все элементы из первого стека во второй (в результате элементы первого стека будут располагаться во втором стеке в порядке, обратном исходному) и вывести адрес новой вершины второго стека. Операции выделения и освобождения памяти не использовать.

Dynamic9. Даны указатели P1 и P2 на вершины двух непустых стеков. Перемещать элементы из первого стека во второй, пока значение вершины первого стека не станет четным (перемещенные элементы первого стека будут располагаться во втором стеке в порядке, обратном исходному). Если в первом стеке нет элементов с четными значениями, то переместить из первого стека во второй все элементы. Вывести адреса новых вершин первого и второго стека (если первый стек окажется пустым, то вывести для него константу nullptr). Операции выделения и освобождения памяти не использовать.

Dynamic10. Дан указатель P1 на вершину непустого стека. Создать два новых стека, переместив в первый из них все элементы исходного стека с четными значениями, а во второй — с нечетными (элементы в новых стеках будут располагаться в порядке, обратном исходному; один из этих стеков может оказаться пустым). Вывести адреса вершин полученных стеков (для пустого стека вывести nullptr). Операции выделения и освобождения памяти не использовать.

Dynamic11. Дан указатель P1 на вершину стека (если стек пуст, то P1 = nullptr). Также дано число N (> 0) и набор из N чисел. Описать тип TStack — запись с одним полем Top типа PNode (поле указывает на вершину стека) — и процедуру Push(S, D), которая добавляет в стек S новый элемент со значением D (S — входной и выходной параметр типа TStack, D — входной параметр целого типа). С помощью процедуры Push добавить в исходный стек данный набор чисел (последнее число будет вершиной стека) и вывести адрес новой вершины стека.

Dynamic12. Дан указатель P1 на вершину стека, содержащего не менее пяти элементов. Используя тип TStack (см. задание Dynamic11), описать функцию Pop(S) целого типа, которая извлекает из стека S первый (верхний) элемент, возвращает его значение и освобождает память, которую занимал извлеченный элемент (S — входной и выходной параметр типа TStack). С помощью функции Pop извлечь из исходного стека пять элементов и вывести их значения. Вывести также указатель на новую вершину стека (если результирующий стек окажется пустым, то этот указатель должен быть равен nullptr).

Dynamic13. Дан указатель P1 на вершину стека. Используя тип TStack (см. задание Dynamic11), описать функции StackIsEmpty(S) логического типа (возвращает True, если стек S пуст, и False в противном случае) и Peek(S) целого типа (возвращает значение вершины непустого стека S, не удаляя ее из стека). В обеих функциях переменная S является входным параметром типа TStack. С помощью этих функций, а также функции Pop из задания Dynamic12, извлечь из исходного стека пять элементов (или все содержащиеся в нем элементы, если их менее пяти) и вывести их значения. Вывести также значение функции StackIsEmpty для результирующего стека и, если результирующий стек не является пустым, значение и адрес его новой вершины.

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ: ОЧЕРЕДЬ

В заданиях данной подгруппы структура «очередь» (queue) моделируется цепочкой связанных узлов-записей типа TNode. Поле Next последнего элемента цепочки равно nullptr. Началом очереди («головой», head) считается первый элемент цепочки, концом («хвостом», tail) — ее последний элемент. Для возможности быстрого добавления в конец очереди нового элемента удобно хранить, помимо указателя на начало очереди, также и указатель на ее конец. В случае пустой очереди указатели на ее начало и конец полагаются равными nullptr. Как и для стека, значением элемента очереди считается значение его поля Data.

Dynamic14. Дан набор из 10 чисел. Создать очередь, содержащую данные числа в указанном порядке (первое число будет размещаться в начале очереди, последнее — в конце), и вывести указатели P1 и P2 на начало и конец очереди.

Dynamic15. Дан набор из 10 чисел. Создать две очереди: первая должна содержать числа из исходного набора с нечетными номерами (1, 3, ..., 9), а вторая — с четными (2, 4, ..., 10);

порядок чисел в каждой очереди должен совпадать с порядком чисел в исходном наборе. Вывести указатели на начало и конец первой, а затем второй очереди.

Dynamic16. Дан набор из 10 чисел. Создать две очереди: первая должна содержать все нечетные, а вторая — все четные числа из исходного набора (порядок чисел в каждой очереди должен совпадать с порядком чисел в исходном наборе). Вывести указатели на начало и конец первой, а затем второй очереди (одна из очередей может оказаться пустой; в этом случае вывести для нее две константы `nullptr`).

Dynamic17. Дано число D и указатели $P1$ и $P2$ на начало и конец очереди (если очередь является пустой, то $P1 = P2 = \text{nullptr}$). Добавить элемент со значением D в конец очереди и вывести новые адреса начала и конца очереди.

Dynamic18. Дано число D и указатели $P1$ и $P2$ на начало и конец очереди, содержащей не менее двух элементов. Добавить элемент со значением D в конец очереди и извлечь из очереди первый (начальный) элемент. Вывести значение извлеченного элемента и новые адреса начала и конца очереди. После извлечения элемента из очереди освободить память, занимаемую этим элементом.

Dynamic19. Дано число N (> 0) и указатели $P1$ и $P2$ на начало и конец непустой очереди. Извлечь из очереди N начальных элементов и вывести их значения (если очередь содержит менее N элементов, то извлечь все ее элементы). Вывести также новые адреса начала и конца очереди (для пустой очереди дважды вывести `nullptr`). После извлечения элементов из очереди освобождать память, которую они занимали.

Dynamic20. Даны указатели $P1$ и $P2$ на начало и конец непустой очереди. Извлекать из очереди элементы, пока значение начального элемента очереди не станет четным, и выводить значения извлеченных элементов (если очередь не содержит элементов с четными значениями, то извлечь все ее элементы). Вывести также новые адреса начала и конца очереди (для пустой очереди дважды вывести `nullptr`). После извлечения элементов из очереди освобождать память, которую они занимали.

Dynamic21. Даны две очереди; адреса начала и конца первой равны $P1$ и $P2$, а второй — $P3$ и $P4$ (если очередь является пустой, то соответствующие адреса равны `nullptr`). Переместить все элементы первой очереди (в порядке от начала к концу) в конец второй очереди и вывести новые адреса начала и конца второй очереди. Операции выделения и освобождения памяти не использовать.

Dynamic22. Дано число N (> 0) и две непустые очереди; адреса начала и конца первой равны $P1$ и $P2$, а второй — $P3$ и $P4$. Переместить N начальных элементов первой очереди в конец второй очереди. Если первая очередь содержит менее N элементов, то переместить из первой очереди во вторую все элементы. Вывести новые адреса начала и конца первой, а затем второй очереди (для пустой очереди дважды вывести `nullptr`). Операции выделения и освобождения памяти не использовать.

Dynamic23. Даны две непустые очереди; адреса начала и конца первой равны $P1$ и $P2$, а второй — $P3$ и $P4$. Перемещать элементы из начала первой очереди в конец второй, пока значение начального элемента первой очереди не станет четным (если первая очередь не содержит четных элементов, то переместить из первой очереди во вторую все элементы). Вывести новые адреса начала и конца первой, а затем второй очереди (для пустой очереди дважды вывести `nullptr`). Операции выделения и освобождения памяти не использовать.

Dynamic24. Даны две непустые очереди; адреса начала и конца первой равны $P1$ и $P2$, а второй — $P3$ и $P4$. Очереди содержат одинаковое количество элементов. Объединить очереди в одну, в которой элементы исходных очередей чередуются (начиная с первого элемента первой очереди). Вывести указатели на начало и конец полученной очереди. Операции выделения и освобождения памяти не использовать.

Dynamic25. Даны две непустые очереди; адреса начала и конца первой равны $P1$ и $P2$, а второй — $P3$ и $P4$. Элементы каждой из очередей упорядочены по возрастанию (в направлении от начала очереди к концу). Объединить очереди в одну с сохранением упорядоченности

элементов. Вывести указатели на начало и конец полученной очереди. Операции выделения и освобождения памяти не использовать, поля Data не изменять.

Dynamic26. Даны указатели P1 и P2 на начало и конец очереди (если очередь является пустой, то $P1 = P2 = \text{nullptr}$). Также дано число N (> 0) и набор из N чисел. Описать тип TQueue — запись с двумя полями Head и Tail типа PNode (поля указывают на начало и конец очереди) — и процедуру Enqueue(Q, D), которая добавляет в конец очереди Q новый элемент со значением D (Q — входной и выходной параметр типа TQueue, D — входной параметр целого типа). С помощью процедуры Enqueue добавить в исходную очередь данный набор чисел и вывести новые адреса ее начала и конца.

Dynamic27. Даны указатели P1 и P2 на начало и конец очереди, содержащей не менее пяти элементов. Используя тип TQueue (см. задание Dynamic26), описать функцию Dequeue(Q) целого типа, которая извлекает из очереди первый (начальный) элемент, возвращает его значение и освобождает память, занимаемую извлеченным элементом (Q — входной и выходной параметр типа TQueue). С помощью функции Dequeue извлечь из исходной очереди пять начальных элементов и вывести их значения. Вывести также адреса начала и конца результирующей очереди (если очередь окажется пустой, то эти адреса должны быть равны nullptr).

Dynamic28. Даны указатели P1 и P2 на начало и конец очереди. Используя тип TQueue (см. задание Dynamic26), описать функцию QueueIsEmpty(Q) логического типа, которая возвращает True, если очередь Q пуста, и False в противном случае (Q — входной параметр типа TQueue). Используя эту функцию для проверки состояния очереди, а также функцию Dequeue из задания Dynamic27, извлечь из исходной очереди пять начальных элементов (или все содержащиеся в ней элементы, если их менее пяти) и вывести их значения. Вывести также значение функции QueueIsEmpty для полученной очереди и новые адреса ее начала и конца.

LISTWORK

ЛЕГКИЕ ЗАДАЧИ НА РАБОТУ С ОДНОСВЯЗНЫМИ ЛИНЕЙНЫМИ СПИСКАМИ

ListWork1. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на второй элемент этого списка P_2 . Известно, что в исходном списке не менее 2 элементов.

ListWork2. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на третий элемент этого списка P_3 . Известно, что в исходном списке не менее 3 элементов.

ListWork3. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на четвертый элемент этого списка P_4 . Известно, что в исходном списке не менее 5 элементов.

ListWork4. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на пятый элемент этого списка P_5 . Известно, что в исходном списке не менее 5 элементов.

ListWork5. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на шестой элемент этого списка P_6 . Известно, что в исходном списке не менее 6 элементов.

ListWork6. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на седьмой элемент этого списка P_7 . Известно, что в исходном списке не менее 7 элементов.

ListWork7. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на восьмой элемент этого списка P_8 . Известно, что в исходном списке не менее 8 элементов.

ListWork8. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на девятый элемент этого списка P_9 . Известно, что в исходном списке не менее 9 элементов.

ListWork9. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на последний элемент этого списка P_x .

ListWork10. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти первый элемент, кратный 4, и вывести указатель на этот элемент списка P_x . Известно, что такой элемент списка точно есть.

ListWork11. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти первый элемент, кратный 5, и вывести указатель на этот элемент списка P_x . Если такого элемента в списке нет, то результат должен быть равен nullptr.

ListWork12. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти второй элемент, кратный 6, и вывести указатель на этот элемент списка P_x . Если такого элемента в списке нет, то результат должен быть равен nullptr.

ListWork13. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти и вывести все элементы списка кратные трем. Известно, что хотя бы один такой элемент в списке точно есть.

ListWork14. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести элементы списка, перечисляя их от последнего к первому.

ListWork15. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести все элементы списка кратные 7 в порядке, обратном исходному. Известно, что в списке есть хотя бы два элемента кратные 7.

ЗАДАЧИ СРЕДНЕЙ СЛОЖНОСТИ НА РАБОТУ С ОДНОСВЯЗНЫМИ ЛИНЕЙНЫМИ СПИСКАМИ

ListWork22. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого второго элемента списка, и вывести ссылку на последний элемент полученного списка P_2 .

ListWork23. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого третьего элемента списка, и вывести ссылку на последний элемент полученного списка P_2 .

ListWork24. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого четвертого элемента списка, и вывести ссылку на последний элемент полученного списка P_2 .

ListWork25. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M перед каждым вторым элементом списка, и вывести ссылку на последний элемент полученного списка P_2 . При нечетном числе элементов исходного списка в конец списка вставлять не надо.

ListWork59. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого K -го элемента списка и вывести ссылку на последний элемент полученного списка P_2 .

ListWork60. Дан односвязный линейный список и указатель на голову списка P_1 . Значения элементов списка упорядочены по возрастанию. Необходимо создать копию исходного списка, после чего во вновь созданном списке вставить в список значение M так, чтобы он остался упорядоченным и вывести ссылку на первый элемент полученного списка P_2 .

ListWork61. Дан текстовый файл, в первой строке которого хранится число N , а во второй строке N целых чисел. Необходимо создать упорядоченный по возрастанию список, в который поместить все эти элементы, при этом очередной элемент вставлять в список так, чтобы не нарушалась его упорядоченность.

ListWork62. Дан текстовый файл, в первой строке которого хранится число N , а во второй строке N целых чисел. Необходимо создать упорядоченный по убыванию список, в который

поместить все эти элементы, при этом очередной элемент вставлять в список так, чтобы не нарушалась его упорядоченность.